Database Management Systems - IT1223

Mr.M.N.M Shakoor, DPS,FAS, University of Vavuniya

MIN, MAX, COUNT, SUM, AVG, LIMIT, IN, BETWEEN, HAVING and GROUP BY.

- Let's create the table named ‘sales’ and insert sales data values from a local text file with tab-separated fields

Location	Product	Price	Sold_at
HQ	Coffee	2	CD and CT
HQ	Coffee	2.5	CD and CT - 01 Hour
Downtown	Bagel	3.5	CD and CT - 02 Hour
Downtown	Coffee	2	CD and CT - 01 Day
HQ	Bagel	1.5	CD and CT - 02 Day
1st Street	Bagel	3.5	CD and CT - 02 Day - 01 Hour
1st Street	Coffee	5	CD and CT - 03 Day
HQ	Bagel	3	CD and CT - 03 Day - 01 Hour

- ***CD*** - Current Date, ***CT*** - Current Time
- For example:

```
+-----------+---------+-------+---------------------+
| location  | product | price | sold_at             |
+-----------+---------+-------+---------------------+
| HQ        | Coffee  |     2 | 2024-03-08 14:54:31 |
```

LIMIT, MIN(), MAX()

- The LIMIT clause is used to specify the number of records to return.
  - `SELECT * FROM sales LIMIT 2;`
- If you want to return only 3 records, start on record 4:
  - `SELECT * FROM sales LIMIT 2 OFFSET 2;`
- Write a query to display the least price of a product in the given sales chart.
  - `SELECT MIN(price) FROM sales;`

LIMIT, MIN(), MAX()

- Write a query to display the highest price of a product in the given sales chart.
  - `SELECT MAX(price) FROM sales;`
- Write a query to display the least priced product and its price in the given sales chart.
  - `SELECT product, MIN(price) FROM sales HAVING MIN(price);`
- Write a query to display the highest priced product and its price in the given sales chart.
  - `SELECT product, MAX(price) FROM sales HAVING MAX(price);`

GROUP BY

- Write a query to display the locations in the sales table: (without duplicates)
  - `SELECT location FROM sales;`
  - `SELECT DISTINCT(location) FROM sales;`
  - ```
    SELECT location
     FROM sales
     GROUP BY location;
    ```
- By grouping on the location column, our database takes these inputs rows and identifies the unique locations among them - these unique locations serve as our "groups.“
    ```

GROUP BY

- Write a query to display the location and product from the sales table according to the location group.

```
SELECT location, product
FROM sales
GROUP BY location;
```

Aggregations (COUNT, SUM, AVG)

- Write a query to display the location and number of sales per location.

```
SELECT
location,
COUNT(*) AS number_of_sales
FROM sales
GROUP BY location;
```

- Here's how the database executes this query:
  - `FROM sales` - First, retrieve all of the records from the sales table
  - `GROUP BY location` - Next, determine the unique location groups
  - `SELECT ...` - Finally, select the location name and the count of the number of rows in that group

Aggregations (COUNT, SUM, AVG)

- Write a query to display the product and number of sales per product.

```
SELECT
product,
COUNT(*) AS number_of_sales
FROM sales
GROUP BY product;
```

- Write a query to display the total amount of money earned from those locations.

```
SELECT
location,
SUM(price) AS total_revenue
FROM sales
GROUP BY location;
```

Aggregations (COUNT, SUM, AVG)

- Write a query to display the finding the average sale price per location.

```
SELECT
location,
AVG(price) AS average_revenue_per_sale
FROM sales
GROUP BY location;
```

Working with multiple groups

- Write a query to display the number of sales per product, per location.

```
SELECT location, product, COUNT(*) AS total_sales
FROM sales
GROUP BY location, product
ORDER BY location, product;
```

- Write a query to find the total number of sales per day.

```
SELECT sold_at, DATE(sold_at) AS sale_date, COUNT(*) AS
sales_per_day
FROM sales
GROUP BY DATE(sold_at)
ORDER BY sold_at;
```

Filtering groups with HAVING

- Write a query to find days where we had more than one sale.

```
SELECT DATE(sold_at) AS sold_date, COUNT(*) AS sales_per_day
FROM sales
WHERE COUNT(*) > 1
GROUP BY DATE(sold_at);
```

1. What is the output?

✓ There is a type of clause that allows us to filter, perform aggregations, and it is evaluated after the GROUP BY clause;

Filtering groups with HAVING

- The HAVING clause is like a WHERE clause for your groups.

```
SELECT DATE(sold_at) AS sold_date, COUNT(*) AS sales_per_day
FROM sales
GROUP BY DATE(sold_at)
HAVING COUNT(*) > 1 ;
```

IN and BETWEEN

- The **IN** operator allows you to specify multiple values in a WHERE clause.
- The **IN** operator is a shorthand for multiple OR conditions.
  - Write a query to display the results where the location is ‘HQ’ or ‘Downtown’ from the sales chart.
    - `SELECT * FROM sales WHERE location IN ('HQ', 'Downtown');`
  - Write a query to display the results where the location is not ‘HQ’ or ‘1st Street’from the sales chart.
    - `SELECT * FROM sales WHERE location NOT IN (‘HQ’, ‘1st Street’);`

IN and BETWEEN

- The **BETWEEN** operator selects values within a given range. The values can be numbers, text, or dates.
- The **BETWEEN** operator is inclusive: begin and end values are included.
  - Write a query to display all the data where the product price range between 1 - 2.5:
    - `SELECT * FROM sales WHERE price BETWEEN 1 AND 2.5;`
  - Write a query to display all the data where the product price range not between 1 - 2.5:
    - `SELECT * FROM sales WHERE price NOT BETWEEN 1 AND 2.5;`

IN and BETWEEN

- Write a query to display all the results from the report where the product price between 1 - 2.5 and product from either ‘HQ’ or ‘Downtown’:

```
SELECT * FROM sales
WHERE price BETWEEN 1 AND 2.5
AND
location IN ('HQ', 'Downtown');
```

To be Continued..!